

Asynchronous Messaging and Queues

Prepared By: Dr. D. P. Singh

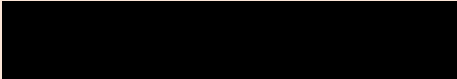
Synchronous Processing

- Synchronous processing means **tasks are executed one after another**. A task must finish before the next one begins. The program waits for each operation to complete.

Example

- Imagine standing in a queue at a bank.
- Person 1 is served.
- Only after Person 1 finishes, Person 2 is served.
- Everyone else waits.

SYNCHRONOUS

Task 1:  Done



Task 2:  Done



Task 3:  Done

Real-life examples

- ATM transaction
- Traditional classroom attendance
- Printing one document after another

Asynchronous Processing

- Asynchronous processing means **multiple tasks can be started without waiting for previous tasks to finish**. While one task is waiting (such as for data from the internet), another task can run.

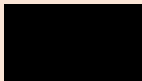
Example

- Imagine ordering food at a restaurant.
- You place your order.
- The chef starts preparing it.
- Instead of waiting at the counter, you sit and chat with friends.
- When your food is ready, your number is called.

ASYNCHRONOUS

Task 1:  Done

Task 2:  Done

Task 3:  Done

(All start together and finish independently.)

Real-life examples

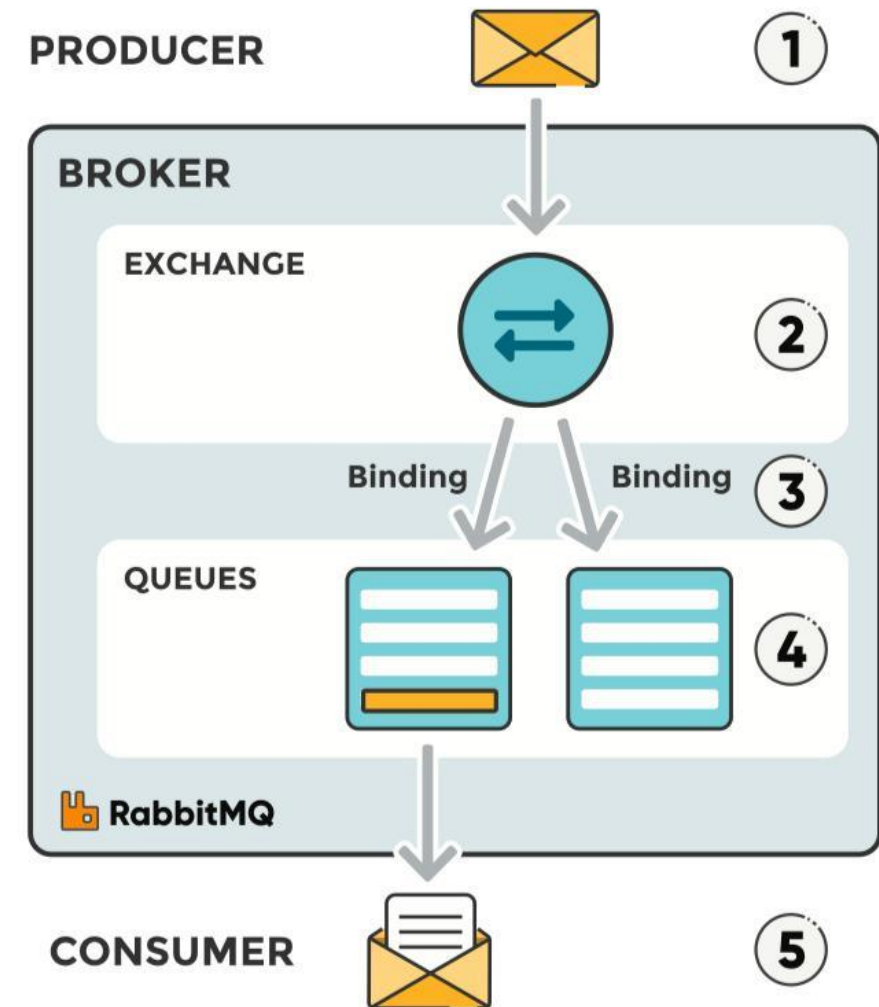
- Downloading a file while browsing the web
- Sending an email while editing a document
- Watching a video while messages arrive

Key Differences

Feature	Synchronous	Asynchronous
Execution	One task at a time	Multiple tasks can progress concurrently
Waiting	Program waits for each task	Program can do other work while waiting
Speed	Slower when tasks involve waiting	Faster for I/O-bound tasks
Complexity	Easier to understand	More complex to write and debug
Best Use	Simple calculations, sequential workflows	Network requests, file operations, databases, web servers

Message Queues (RabbitMQ)

- A **message queue** is a communication mechanism where one application (**Producer**) sends messages to a queue, and another application (**Consumer**) retrieves and processes them.
- The producer and consumer **do not need to communicate directly**. This is called **decoupling**.

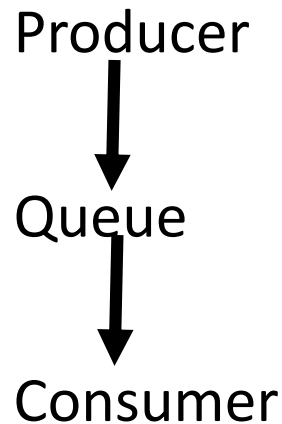


Why Decouple Producer and Consumer?

- Without a queue:



- If the consumer crashes,
 - Producer fails
 - Data may be lost
 - System becomes tightly coupled
- With RabbitMQ:



- The producer simply places messages into the queue and continues working. The consumer processes messages whenever it is available.

E-commerce Example

User clicks "Buy"



RabbitMQ Queue:

Payment Service
Inventory Service
Email Service

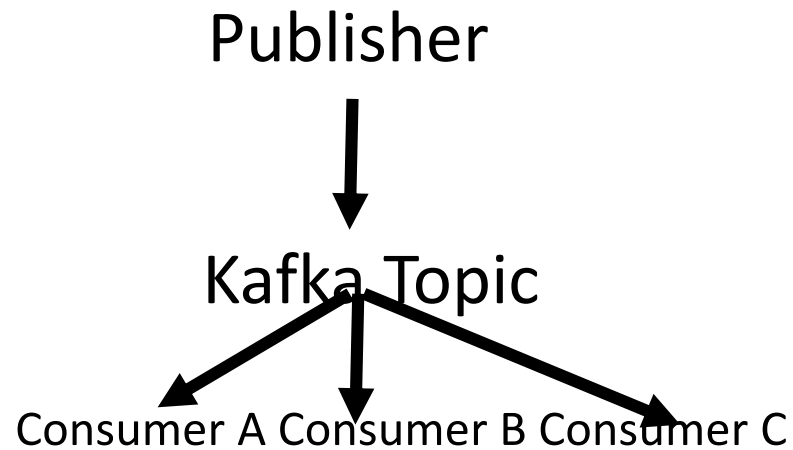
- Even if the email service is temporarily down, payment and inventory processing continue.

Advantages

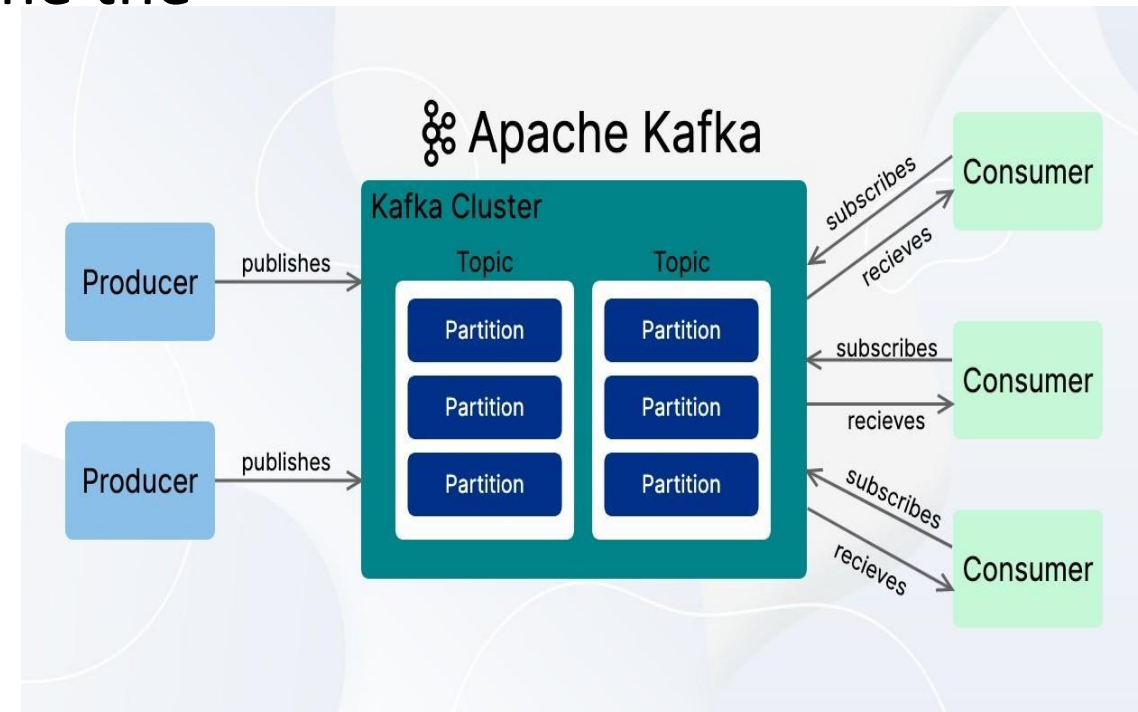
- Loose coupling
- Reliable communication
- Fault tolerance
- Load balancing
- Easy scalability

Publish/Subscribe (Pub/Sub) Model (Kafka)

- Unlike RabbitMQ, **Kafka** is commonly used as an event streaming platform where publishers broadcast events to **topics**, and multiple subscribers can independently consume the same events.

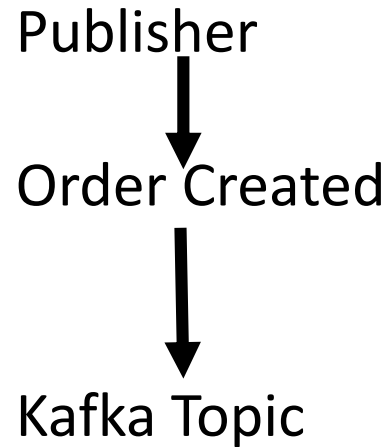


- This is called **Fan-Out Architecture**.
- One published event can be delivered to many independent consumers.



Example

- A customer places an order.



- Billing Service
 - Inventory Service
 - Analytics Service
 - Notification Service
 - Recommendation Engine
- Each service receives the same event independently.

Social Media Example

- When a celebrity uploads a post:



Followers
Follower 1
Follower 2
Follower 3
Follower 4
Follower 5

- One event reaches thousands or millions of subscribers.

RabbitMQ vs Kafka

RabbitMQ	Kafka
Queue-based messaging	Event streaming platform
One consumer typically processes each message	Multiple consumers can read the same event independently
Best for task processing	Best for event streaming and analytics
Messages are removed after acknowledgment	Events are retained for a configurable period
Common use cases: background jobs, order processing	Common use cases: logs, metrics, real-time pipelines

Handling Backpressure

Backpressure:

- Backpressure occurs when **producers generate messages faster than consumers can process them**, causing queues or topics to grow.

- **Example**

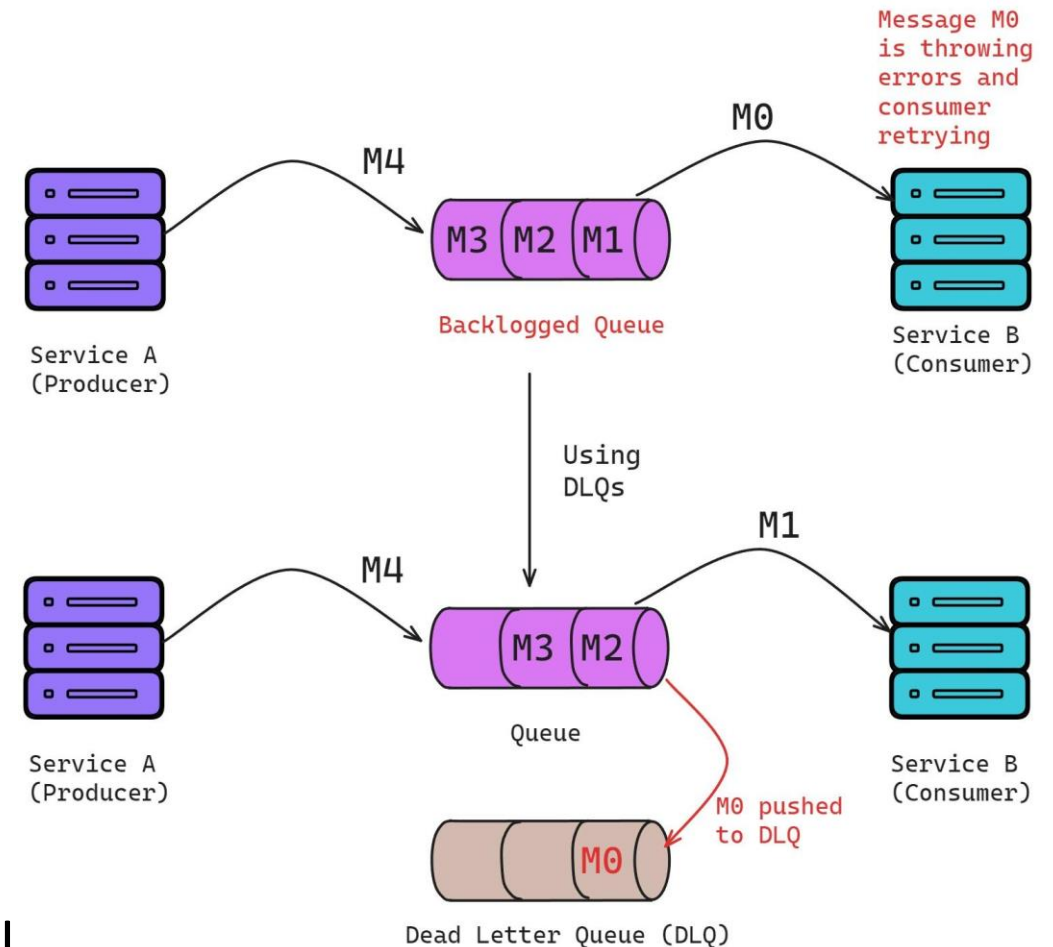
Suppose:

- Producer: 1000 messages/sec
- Consumer: 100 messages/sec
- Queue: 900 messages added every sec

Eventually memory fills up and response times increase.

Ways to Handle Backpressure

- Add More Consumers
- Rate Limiting
 - Slow down producers.
- Increase Queue Capacity
- Batch Processing
 - Instead of processing one message at a time,
 - Read: 100 messages and Process together
- Dead-Letter Queue (DLQ)
 - Sometimes messages **cannot be processed successfully**, even after several retries.



Ways to Handle Backpressure

- **Dead-Letter Queue (DLQ)**
- Examples:
 - Invalid data
 - Corrupted message
 - Database permanently unavailable
 - Missing required fields
- Instead of retrying forever, the message is moved to a **Dead-Letter Queue (DLQ)** for later inspection.

Benefits of DLQ

- Prevents infinite retry loops
- Keeps the main queue flowing
- Makes debugging easier
- Preserves failed messages for investigation
- Improves overall system reliability

Thank You